

Interpréteur INF404 — Rapport de projet

L2 Informatique — Université Grenoble Alpes — 2025/2026

1. Langage accepté

Le langage accepté est un langage impératif simple inspiré du pseudo-code des slides INF404. Il supporte les **expressions arithmétiques** (entiers et flottants, opérateurs $+$ $-$ $\times$ $\div$, parenthèses), les **variables** (identificateurs), les **instructions** : affectation ($x = \text{expr}$), lecture ($\text{lire}(x)$), écriture ($\text{ecrire}(\text{expr})$), la **structure conditionnelle** *si cond alors ... sinon ... fsi* (branche sinon optionnelle), et la **boucle** *tanque cond faire ... fait*. Les instructions sont séparées par des point-virgules. Les opérateurs de comparaison disponibles sont : $<$ $>$ $\leq$ $\geq$ $==$ $!=$.

2. Traitement effectué

Le traitement se déroule en trois phases successives. **Analyse lexicale** : le texte source est découpé en lexèmes (mots-clés, identificateurs, nombres, symboles). **Analyse syntaxique** : un analyseur descendant récursif construit un **arbre abstrait (AST)** fidèle aux grammaires des slides — les nœuds `N_AFF`, `N_LIRE`, `N_ECRIRE`, `N_SEPINST`, `N_IF`, `N_WHILE`, `N_IDF`, `VALEUR`, `OPERATION`, et les comparaisons (`N_INF`, `N_SUP`...) sont des nœuds à part entière. **Interprétation** : l'AST est parcouru récursivement ; les variables sont stockées dans une table des symboles (tableau de paires nom/valeur). Des erreurs lexicales, syntaxiques et sémantiques (variable non déclarée) sont détectées avec indication de la ligne et de la colonne. Le projet est structuré en modules indépendants (lexical, syntaxique, ast, symbole) compilés via un Makefile.

3. Exemples de programmes

Exemple 1 — Condition si/sinon

```
x = 5 ;
y = 1 ;
si x < 10 alors
    x = x + 1
sinon
    y = y * 2 ;
fsi ;
ecrire(x) ;
ecrire(y)
```

Sortie :

```
6
1
Table des symboles :
x = 6 | y = 1
```

Exemple 2 — Conditions imbriquées

```
x = 5 ; y = 10 ;
si x < y alors
    si x < 3 alors écrire(0)
    sinon écrire(1) fsi
sinon
    écrire(2)
fsi ;
```

Sortie :

```
1
Table des symboles :
x = 5 | y = 10
```

Exemple 3 — Boucle tanque (somme de 1 à 5)

```
i = 1 ;
somme = 0 ;
tanque i <= 5 faire
    somme = somme + i ;
    i = i + 1
fait ;
ecrire(somme)
```

Sortie :

```
15
Table des symboles :
i = 6 | somme = 15
```

4. Conclusion

L'interpréteur est entièrement fonctionnel : il accepte des programmes impératifs avec affectations, entrées/sorties, conditions imbriquées et boucles, conformément aux grammaires des slides INF404. La séparation en modules indépendants (lexical, syntaxique, AST, symboles) rend le projet facilement extensible — on pourrait y ajouter des fonctions, des tableaux ou un typage statique sans réécrire l'existant.